

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería

**Manual Técnico - Sistema de Gestión de Restaurante
(SGR)**

Integrantes:

Carnet	Nombre
201731523	Byron Fernando Torres Ajxup
201930566	Alexander Tzoc Alvarado
201830530	Wilson Jonatan Chay Santizo

Fecha: 17 de septiembre de 2026

Sistema de Gestión de Restaurante (SGR)

Repositorios: sgr-api, sgr-admin-client, sgr-ops-client

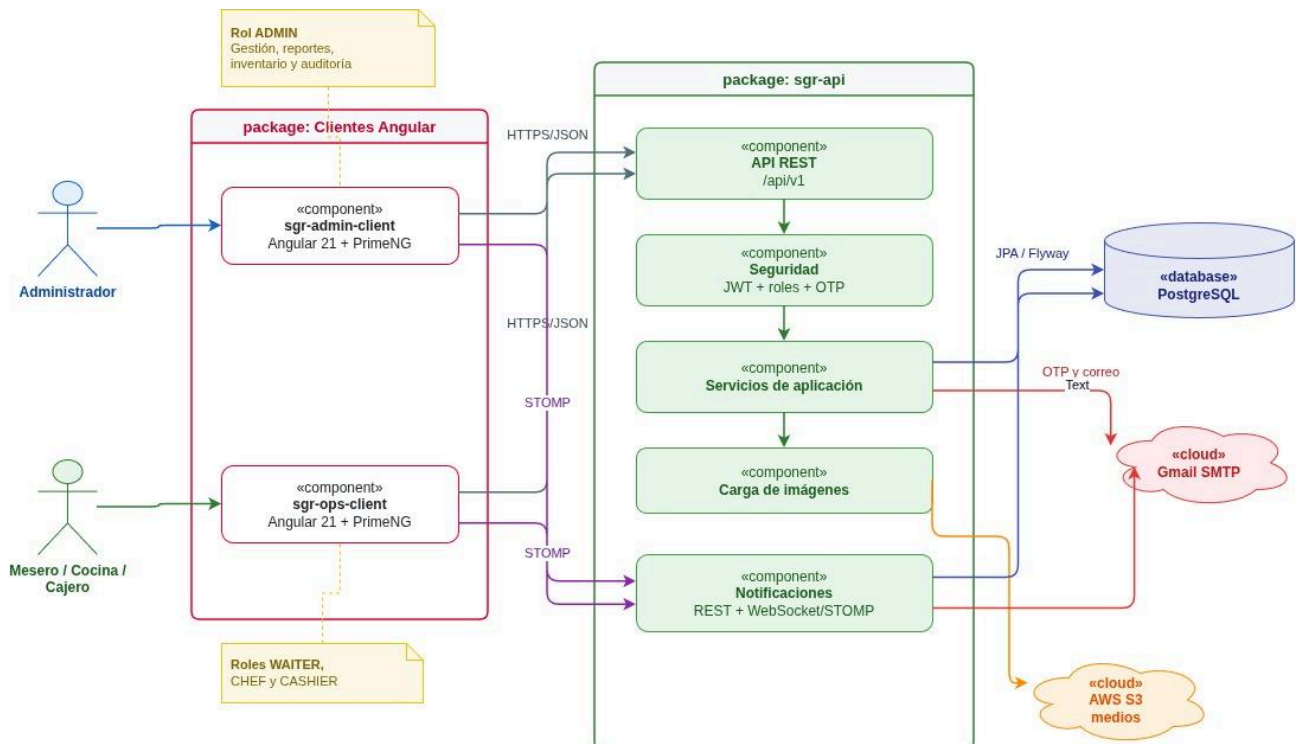
Este documento describe la arquitectura, configuración, operación y despliegue del sistema. No sustituye la guía de despliegue CI/CD; resume únicamente los pasos y decisiones técnicas que deben conocerse para mantener la solución.

1. Alcance y visión general

SGR está compuesto por tres aplicaciones que trabajan sobre el mismo dominio de restaurante:

- **API:** backend Spring Boot que centraliza autenticación, autorización, reglas de negocio, persistencia, facturación, inventario, reportes y notificaciones.
- **Cliente administrativo:** aplicación Angular para usuarios con rol ADMIN.
- **Cliente operativo:** aplicación Angular para WAITER, CHEF y CASHIER.

Los clientes no acceden directamente a PostgreSQL. Todas las operaciones pasan por la API HTTP bajo /api/v1 y, para notificaciones en tiempo real, por WebSocket/STOMP.



2. Tecnologías

Capa	Tecnología
Backend	Java, Spring Boot, Spring MVC, Spring Data JPA, Spring Security
Persistencia	PostgreSQL, Flyway, Hibernate/JPA
Contratos	DTOs, validación Spring, RFC 9457 para errores, OpenAPI
Autenticación	JWT Bearer, OTP por correo, 2FA opcional
Tiempo real	WebSocket/STOMP con destino por usuario
Frontends	Angular 21, TypeScript, PrimeNG, Tailwind CSS, Transloco
Pruebas	JUnit/Spring Test en API; Vitest mediante Angular CLI en clientes
Imágenes	AWS S3 mediante S3FileUploadService
Automatización	Gradle Wrapper, npm, Docker, Jenkins
Producción	AWS EC2, RDS PostgreSQL, ECR, S3, SSM Parameter Store

3. Repositorios y responsabilidades

3.1 sgr-api

Ubicación de la API en `sgr-api/src/main/java/com/sgr`:

```

None
application/      Casos de uso y servicios de aplicación
config/           Configuración, propiedades, OpenAPI y bootstrap
domain/model/     Entidades y enumeraciones del dominio
domain/repository Repositorios JPA
exception/        Excepciones y respuestas problem+json
security/         JWT, filtros y autorización
web/             Controladores y DTOs HTTP

```

Las migraciones Flyway están en `src/main/resources/db/migration/`. Son forward-only: una migración aplicada no debe editarse; los cambios de esquema se agregan en una nueva migración.

3.2 sgr-admin-client

Cliente Angular exclusivo para ADMIN. Incluye administración de usuarios, mesas, configuración, insumos, inventario, platillos, recetas, combos, reportes, historial de facturas y notificaciones.

- Sesión almacenada en localStorage con la clave sgr-admin.auth.session.
- Interceptor agrega Authorization: Bearer <token>.
- Proxy local: /api hacia http://localhost:8090.
- Build generado en dist/sgr-admin-client/.

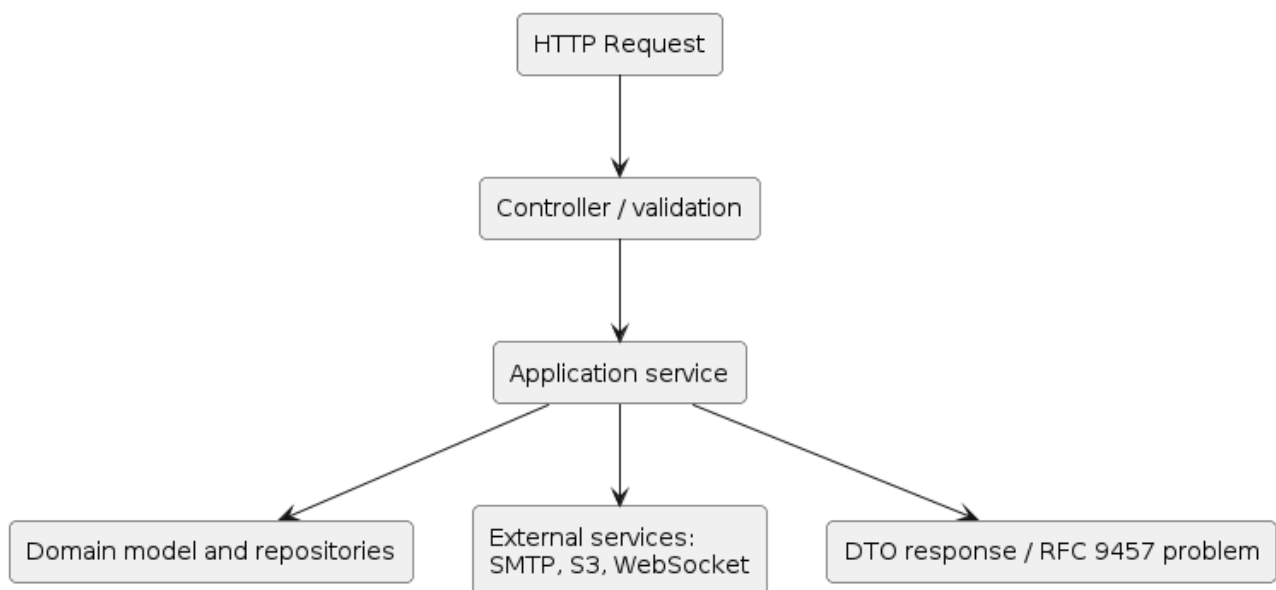
3.3 sgr-ops-client

Cliente Angular para la operación diaria del restaurante:

- Meseros: mesas, cuentas, órdenes y seguimiento de servicio.
- Cocina: cola y estados de preparación.
- Cajeros: caja, cobros, emisión y consulta de comprobantes.
- Sesión almacenada en localStorage con la clave sgr-ops.auth.session.
- Proxy local: /api hacia http://localhost:8090.
- Build generado en dist/sgr-ops-client/.

4. Arquitectura interna de la API

La API utiliza una arquitectura por capas:



4.1 Controladores y DTOs

Los controladores de web/ reciben parámetros HTTP, aplican autorización declarativa y delegan los casos de uso. Los DTOs evitan exponer directamente las entidades JPA y establecen el contrato que consumen ambos frontends.

Las respuestas de error siguen el formato application/problem+json, con campos como type, title, status, detail, instance y code.

4.2 Seguridad

- La autenticación produce JWT cuando la cuenta puede completar el flujo requerido.
- El filtro JWT obtiene el usuario autenticado para cada petición protegida.
- @PreAuthorize restringe operaciones por rol.
- El cliente administrativo usa endpoints de login administrativo.
- Los roles principales son ADMIN, WAITER, CHEF y CASHIER.
- Un cajero solo consulta sus propias facturas; un administrador consulta el historial completo.
- Las contraseñas, OTP, tokens y credenciales SMTP no deben aparecer en logs.

4.3 OTP, correo y 2FA

Los desafíos OTP se almacenan de forma segura y tienen expiración, límite de intentos y cooldown de reenvío. Gmail SMTP se abstrae detrás del servicio de correo. En pruebas automatizadas se usa un emisor de prueba, por lo que no se requieren credenciales reales.

4.4 Notificaciones

La publicación de una notificación y su registro de outbox se realizan en la misma transacción. El cliente recibe:

- REST: GET /api/v1/notifications, unread-count, marcar una o todas como leídas.
- STOMP: /api/v1/ws/notifications.
- Suscripción por usuario: /user/queue/notifications.

REST es la fuente de verdad después de una reconexión. El cliente mantiene fallback polling cuando el WebSocket no está disponible.

5. Persistencia y modelo de datos

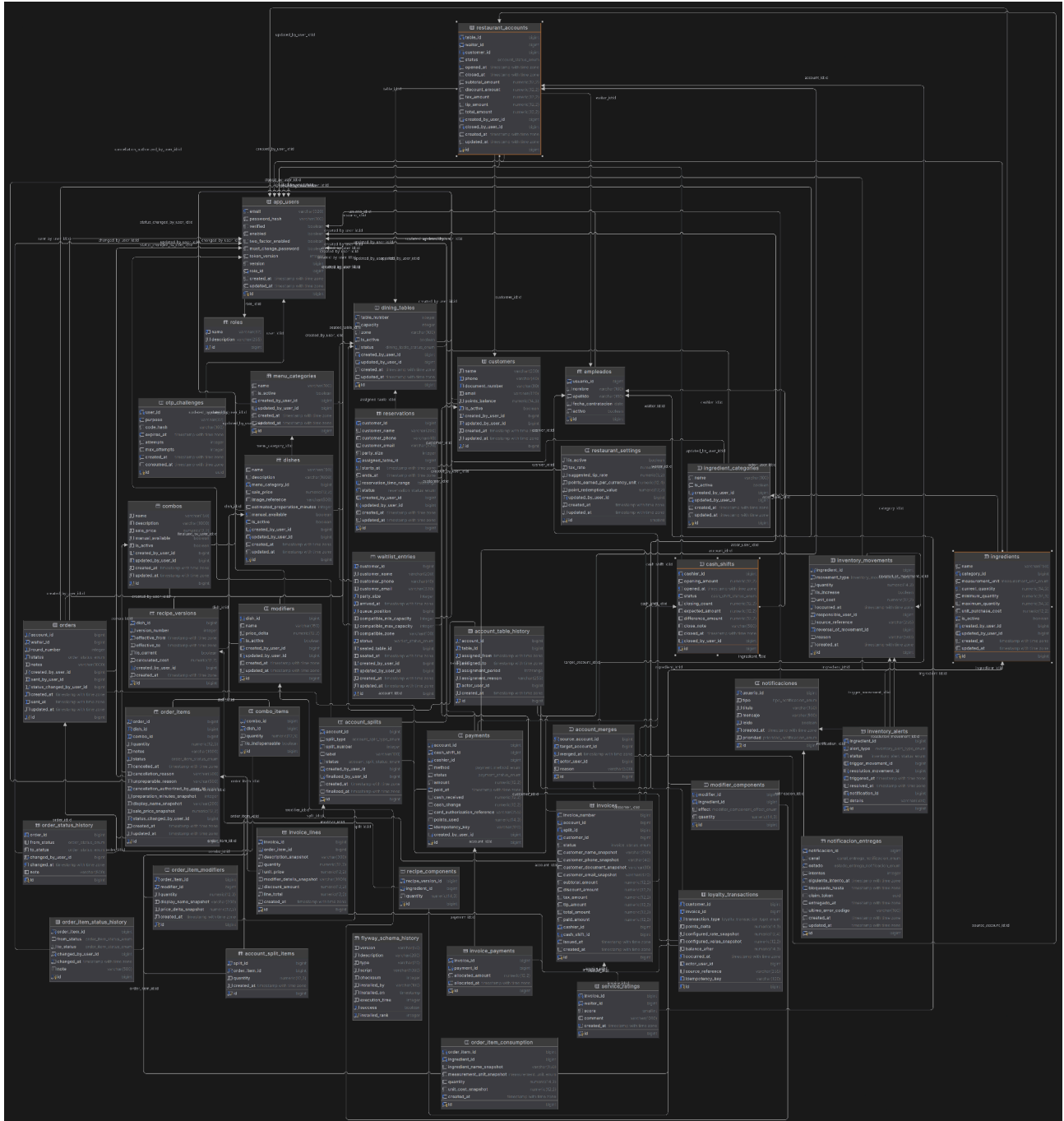
PostgreSQL almacena autenticación, empleados, catálogo, inventario, mesas, cuentas, órdenes, caja, pagos, facturas, clientes y notificaciones. El esquema se crea y actualiza con Flyway.

Relaciones relevantes:

- app_users referencia roles y puede tener un perfil en empleados.
- restaurant_accounts relaciona mesa, mesero, cliente y totales.
- orders y order_items representan comandas y líneas históricas.
- cash_shifts, payments e invoices representan caja y facturación.
- invoice_lines e invoice_payments conservan snapshots para que el comprobante no dependa de cambios posteriores del catálogo.

- dishes, ingredients, recetas, modificadores y combos forman el catálogo e inventario.
- notificaciones y notificacion_entregas soportan la bandeja y el outbox.

5.1 Diagrama ER



6. Configuración local

6.1 API

Requisitos recomendados:

- Docker y Docker Compose.

- JDK 25 para coincidir con build.gradle y Jenkins.
- Variables de entorno definidas; Spring y Gradle no cargan .env automáticamente, aunque el proyecto lee el archivo desde las tareas configuradas.

Iniciar PostgreSQL:

Shell

```
cd sgr-api  
docker compose up -d
```

Ejecutar la API:

Shell

```
set -a; source .env; set +a  
./gradlew bootRun
```

En desarrollo:

- API: <http://localhost:8090/api/v1>
- PostgreSQL: localhost:5435
- OpenAPI: <http://localhost:8090/api/v1/v3/api-docs>

Comandos de verificación:

Shell

```
./gradlew compileJava  
./gradlew test  
./gradlew check  
./gradlew build
```

6.2 Clientes Angular

Shell

```
cd sgr-admin-client  
npm ci  
npm start
```

Shell

```
cd sgr-ops-client
```

```
npm ci
```

```
npm start
```

Por defecto, el cliente admin usa el puerto 4200 y el cliente ops el puerto 4201. Ambos proxies apuntan al API local.

Nota de compatibilidad: Los README de los clientes indican Node.js 24, mientras los jobs de Jenkins utilizan NodeJS 22 LTS. Para reproducir CI debe usarse la versión definida por Jenkins; si se actualiza la versión local, conviene mantener sincronizados README, lockfiles y agentes.

7. Variables de entorno

Variables esenciales del API:

Variable	Uso
DATABASE_URL	URL JDBC de PostgreSQL
DATABASE_USERNAME / DATABASE_PASSWORD	Credenciales de base de datos
SECRET_KEY_JWT	Firma JWT, mínimo 32 caracteres
INITIAL_ADMIN_EMAIL / INITIAL_ADMIN_PASSWORD	Administrador inicial
EMAIL_SENDER_APP / EMAIL_SENDER_PASSWORD	Gmail SMTP
AWS_REGION / S3_BUCKET	Almacenamiento de imágenes
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credenciales AWS locales; en producción se prefiere rol IAM
NOTIFICATIONS_WEBSOCKET_ALLOWED_ORIGINS	Orígenes permitidos para STOMP

Los secretos de producción se mantienen en AWS Systems Manager Parameter Store como SecureString, bajo `/sgr/prod/api/`. No deben commitearse archivos `.env` con valores reales.

8. Pruebas y calidad

API

Shell

```
./gradlew clean build -PunitTestsOnly
```

El pipeline CI excluye las pruebas de integración y SgrApiApplicationTests mediante unitTestsOnly. Las pruebas de integración requieren PostgreSQL activo y se ejecutan manualmente cuando corresponde.

Frontends

Shell

```
npm ci  
npm test -- --watch=false  
npm run build
```

El cliente ops limita la concurrencia de Vitest en Jenkins mediante vitest.config.ts para evitar agotamiento de workers en agentes compartidos. Un resultado exitoso debe terminar sin errores de workers y con todas las pruebas reportadas.

9. CI/CD y despliegue AWS

El flujo de ramas es:

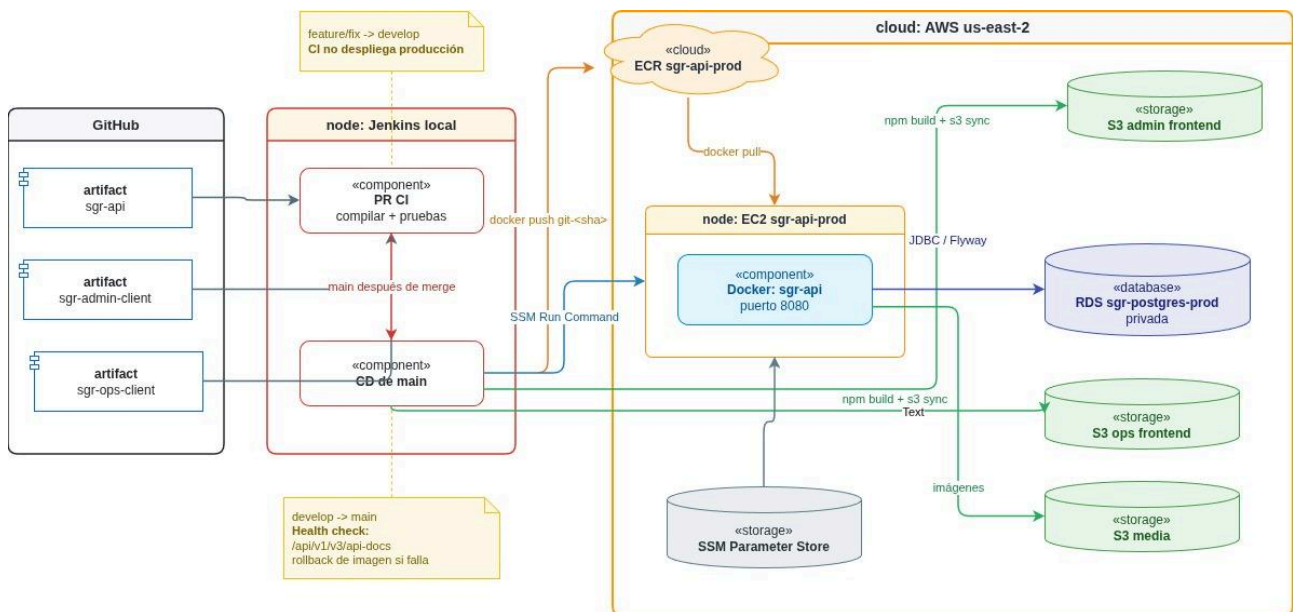
None

```
feature/* o fix/* -> PR develop -> PR main -> CD producción
```

Los PR validan compilación y pruebas; no despliegan producción. Los pipelines CD toman main.

9.1 Componentes de producción

- API Spring Boot en Docker sobre EC2.
- PostgreSQL privado en RDS.
- Imágenes versionadas por commit en ECR.
- Frontends estáticos en buckets S3.
- Imágenes de platillos en un bucket S3 de medios.
- Secretos en Parameter Store.
- Jenkins usa IAM limitado y SSM para ejecutar el despliegue remoto.



9.2 Despliegue del API

El pipeline Jenkinsfile.cd del API:

1. Obtiene main y calcula el SHA.
2. Construye y etiqueta la imagen git-<sha>.
3. Publica la imagen en ECR.
4. Envía un script comprimido a EC2 mediante SSM.
5. Lee parámetros de producción desde Parameter Store.
6. Arranca el contenedor en el puerto 8080.
7. Comprueba /api/v1/v3/api-docs.
8. Restaura la imagen anterior si el contenedor nuevo no queda saludable.

9.3 Despliegue de clientes

Cada cliente ejecuta npm ci, pruebas unitarias, build y aws s3 sync --delete. index.html se publica sin caché y el bucket usa index.html como documento de error para soportar rutas de Angular.

9.4 Componentes AWS documentados

La configuración descrita en la guía de despliegue usa la región us-east-2:

- ECR: sgr-api-prod.
- EC2: sgr-api-prod.
- RDS: sgr-postgres-prod, sin acceso público.
- S3 admin: sgr-admin-frontend-737404990785.
- S3 ops: sgr-ops-frontend-737404990781.

- S3 medios: sgr-media-737404990785.

La IP pública de EC2 puede cambiar si no es Elastic IP. Si cambia, deben actualizarse los `environment.production.ts` de ambos clientes y desplegarse de nuevo.

10. Verificación posterior al despliegue

1. OpenAPI responde HTTP 200.
2. Los dos sitios S3 cargan `index.html`.
3. Admin inicia sesión y completa OTP si está habilitado.
4. Un usuario operativo inicia sesión en Ops.
5. Se prueban mesas, órdenes, cocina y caja.
6. Se emite una factura y se visualiza el comprobante.
7. Se verifica una notificación en tiempo real y su fallback REST.
8. Los jobs CD terminan en Finished: SUCCESS.

Comprobación básica desde EC2:

Shell

```
docker ps --filter name=sgr-api  
curl -i http://127.0.0.1:8080/api/v1/v3/api-docs | head
```

11. Capturas técnicas recomendadas

A continuación se presentan los registros y evidencias fotográficas de los componentes desplegados en el entorno de integración y producción:

1. **Pipelines de CI/CD en Jenkins:** Ejecución exitosa (Finished: SUCCESS) en las etapas de compilación, pruebas y despliegue continuo.
 1. **Cliente Administrativo (sgr-admin-client):** Publicación correcta en S3.

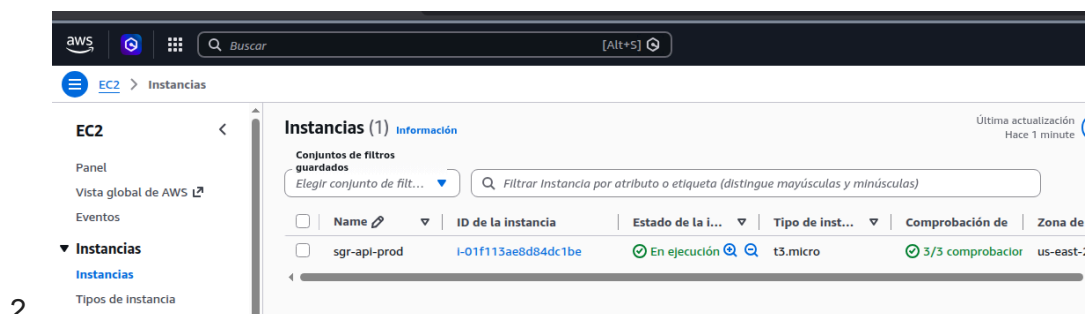
```
Jenkins / sgr-admin-client-cd / #13
inglés español
upload: dist/sgr-admin-client/browser/chunk-WD2I4I4M.js to s3://sgr-admin-frontend-737404990785/chunk-WD2I4I4M.js
Completed 3.5 MiB/3.7 MiB (176.7 KiB/s) with 1 file(s) remaining
upload: dist/sgr-admin-client/browser/chunk-ETV3LNF2.js to s3://sgr-admin-frontend-737404990785/chunk-ETV3LNF2.js
+ aws s3 cp dist/sgr-admin-client/browser/index.html s3://sgr-admin-frontend-737404990785/index.html --region us-east-2 --cache-control no-cache,no-store,must-revalidate --
metadata-directive REPLACE
Completed 7.7 KiB/7.7 KiB (567 Bytes/s) with 1 file(s) remaining
upload: dist/sgr-admin-client/browser/index.html to s3://sgr-admin-frontend-737404990785/index.html
+ aws s3 cp dist/sgr-admin-client/browser/index.html s3://sgr-admin-frontend-737404990785/error.html --region us-east-2 --cache-control no-cache,no-store,must-revalidate --
metadata-directive REPLACE
Completed 7.7 KiB/7.7 KiB (2.5 KiB/s) with 1 file(s) remaining
upload: dist/sgr-admin-client/browser/index.html to s3://sgr-admin-frontend-737404990785/error.html
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
Guardando archivos
Almacenando firmas
[Pipeline] deleteDir
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

2. API Backend (sgr-api): Despliegue remoto exitoso en servidor EC2 mediante SSM.

```
Jenkins / sgr-api-cd / #13
inglés español
+ sleep 3
+ aws ssm get-command-invocation --region us-east-2 --command-id 08bfd14b-00f7-4e7b-ae19-dd6e33387ba0 --instance-id i-01f113ae8d84dc1be --query Status --output text
+ status=InProgress
+ sleep 3
+ aws ssm get-command-invocation --region us-east-2 --command-id 08bfd14b-00f7-4e7b-ae19-dd6e33387ba0 --instance-id i-01f113ae8d84dc1be --query Status --output text
+ status=InProgress
+ sleep 3
+ aws ssm get-command-invocation --region us-east-2 --command-id 08bfd14b-00f7-4e7b-ae19-dd6e33387ba0 --instance-id i-01f113ae8d84dc1be --query Status --output text
+ status=Success
+ echo Remote deployment completed successfully.
Remote deployment completed successfully.
+ exit 0
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ [ -n 737404990785.dkr.ecr.us-east-2.amazonaws.com/sgr-api-prod:git:38fd00fe875d ]
+ docker image rm 737404990785.dkr.ecr.us-east-2.amazonaws.com/sgr-api-prod:git:38fd00fe875d
Untagged: 737404990785.dkr.ecr.us-east-2.amazonaws.com/sgr-api-prod:git:38fd00fe875d
Untagged: 737404990785.dkr.ecr.us-east-2.amazonaws.com/sgr-api-prod@sha256:9a52de4675e4a3c1d51d8ad3f74801bd7c9ab19a44f86561e4b69341e2c98b61
Deleted: sha256:1859592d5ebddcf7617b33ab8a8ff63c7c140118abd8ae8825a1cd9404bd6f4da
[Pipeline] deleteDir
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

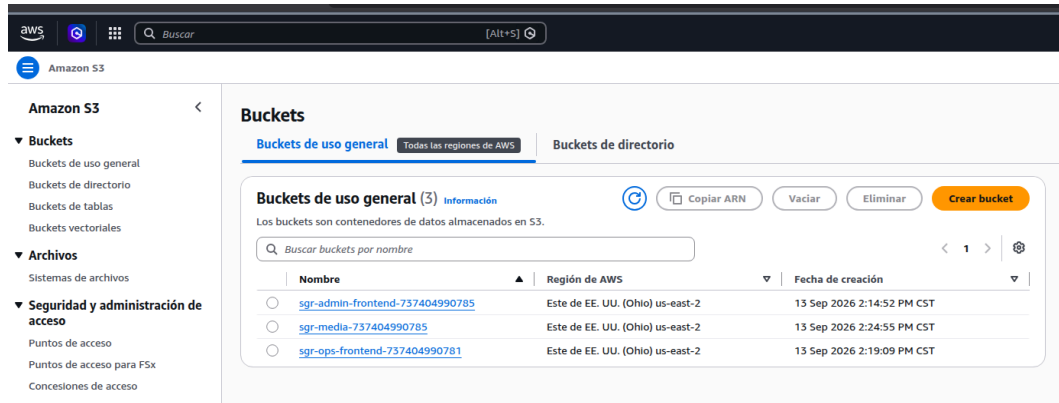
2. Infraestructura y Servicios en AWS:

1. Amazon EC2: Estado de la instancia en ejecución (sgr-api-prod) en la región us-east-2.

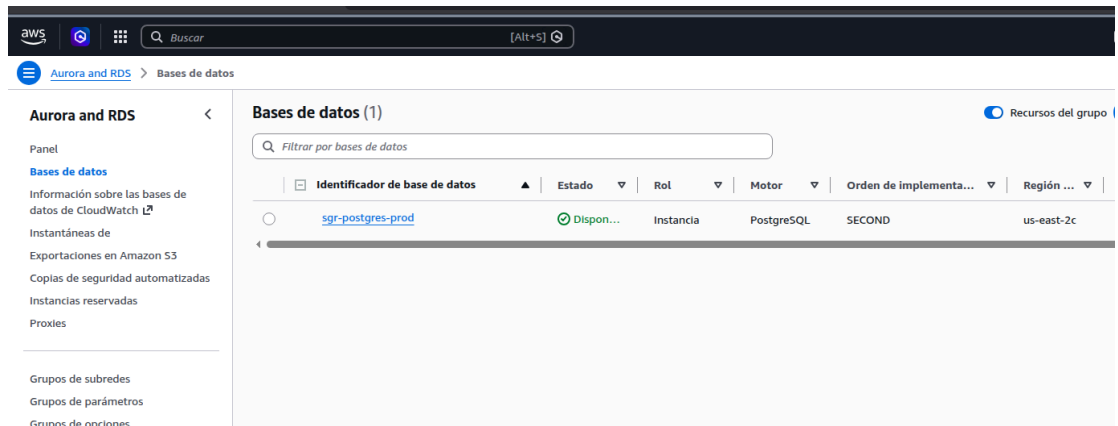


2.

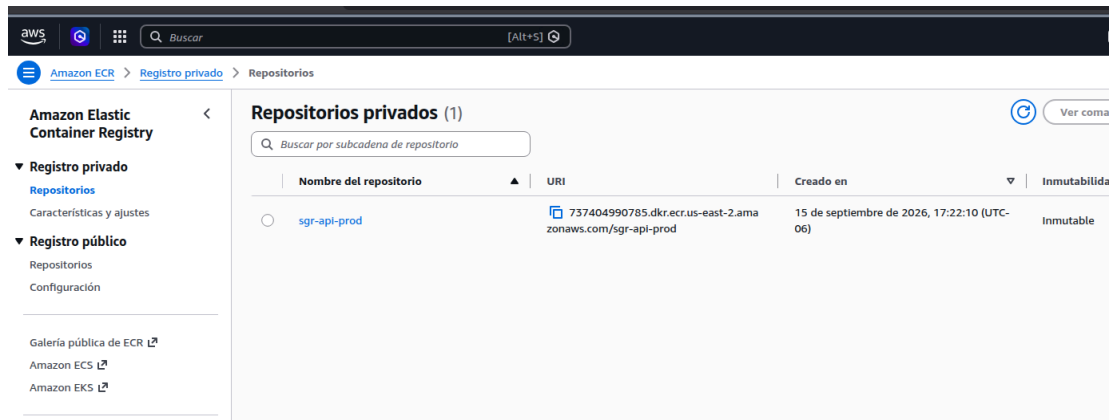
3. Amazon S3: Listado de los buckets de almacenamiento configurados (sgr-admin-frontend-737404990785, sgr-ops-frontend-737404990781 y sgr-media-737404990785).



- 4.
5. Amazon RDS: Instancia de base de datos PostgreSQL privada (sgr-postgres-prod).



- 6.
7. Amazon ECR: Repositorio de imágenes Docker sgr-api-prod con las etiquetas de versión por commit (git-<sha>).



- 8.
9. AWS Systems Manager (SSM) Parameter Store: Lista de parámetros de configuración de producción bajo la ruta /sgr/prod/api/ (asegurando ocultar los valores sensibles).

Mis parámetros

☐	Nombre	Capa	Tipo	Última modificación
☐	/sgr/prod/api/AWS_REGION	Estándar	String	Tue, 15 Sep 2026 21:32:45 GMT
☐	/sgr/prod/api/DATABASE_PASSWORD	Estándar	SecureString	Tue, 15 Sep 2026 20:17:31 GMT
☐	/sgr/prod/api/DATABASE_URL	Estándar	String	Tue, 15 Sep 2026 20:09:11 GMT
☐	/sgr/prod/api/DATABASE_USERNAME	Estándar	String	Tue, 15 Sep 2026 20:11:55 GMT
☐	/sgr/prod/api/EMAIL_SENDER_APP	Estándar	String	Tue, 15 Sep 2026 20:25:38 GMT
☐	/sgr/prod/api/EMAIL_SENDER_PASSWORD	Estándar	SecureString	Thu, 17 Sep 2026 23:20:45 GMT
☐	/sgr/prod/api/INITIAL_ADMIN_EMAIL	Estándar	String	Tue, 15 Sep 2026 21:29:22 GMT
☐	/sgr/prod/api/INITIAL_ADMIN_PASSWORD	Estándar	SecureString	Tue, 15 Sep 2026 21:31:48 GMT
☐	/sgr/prod/api/NOTIFICATIONS_WEBSOCKET_ALLOWED_ORIGINS	Estándar	String	Wed, 16 Sep 2026 04:54:10 GMT
☐	/sgr/prod/api/S3_BUCKET	Estándar	String	Tue, 15 Sep 2026 21:33:33 GMT
☐	/sgr/prod/api/SECRET_KEY_JWT	Estándar	SecureString	Tue, 15 Sep 2026 20:21:17 GMT
☐	/sgr/prod/api/SPRING_PROFILES_ACTIVE	Estándar	String	Tue, 15 Sep 2026 21:34:50 GMT

12. Estimación con Planning Poker y Escala de Fibonacci

Para la estimación del esfuerzo de desarrollo en los repositorios del sistema SGR (sgr-api, sgr-admin-client y sgr-ops-client), el equipo de desarrollo adopta la técnica de Planning Poker utilizando la secuencia modificada de Fibonacci (0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100).

- **Puntos de Historia (Story Points):** Representan una medida relativa del esfuerzo total, complejidad técnica y riesgos asociados a una historia de usuario, en lugar de horas absolutas.
- **Criterios de referencia para SGR:** Historias de menor complejidad (como ajustes menores en interfaces de Angular o nuevos DTOs de lectura en Spring Boot) se estiman en valores bajos (1-3 pts), mientras que integraciones complejas (como WebSocket/STOMP, flujos 2FA o transacciones de caja y facturación) requieren estimaciones más elevadas (5-13 pts).
- **Descomposición de tareas:** Historias estimadas en 20 o más puntos deben dividirse en historias de usuario más pequeñas antes de ser incluidas en la planificación del Sprint.